

// phase 0 through 4 · the white-label product build

Powered Platform Build Plan

Powered Platform - Build Plan

The white-label product sold to founding partners. Internal. June 2026.

The product: a branded instance of the Body Recode platform for another practitioner, powered by the BR interpretation engine, delivered through Studio of Ten. Capped at ten founding partners.

01 1. Product model (locked this session)

- **Layer 1 = the interpretation engine.** Reads the client and produces the interpretation (scorecard interpretation, CFFS, Fat Map, foundational/program/nutrition/trajectory readings). BR doctrine.
- **Layer 2 = the coaching application.** Everything anyone touches: the scorecard funnel, client portal, intake, programs, nutrition, check-ins, coach dashboard. Consumes Layer 1.
- **Engine = diagnosis, coach = prescription.** The engine reads and suggests. The coach keeps full prescriptive authority: coach-guidance override is built, plans are coach-editable, the coach can steer or replace the engine's output.
- **Hard safety floors stay hard.** RRS clamps, Fat Map limits, injury contraindications, eligibility floors cannot be overridden, by coach or partner. This is the "powered by Body Recode" quality and liability shield.
- **Everything is coach-gated.** Engine generates, coach reviews/edits/approves/publishes, client sees. No engine output reaches a partner's client unreviewed.
- **Doctrine: (A) now, (B) later.** (A) partners run BR's doctrine, branded as theirs, with their own prescriptions on top. (B) each partner's method injected into the engine, reserved for after the first ten.

02 2. Pricing (locked)

Per-seat, **not** revenue-share: 1. **Setup** - one-time, stands the platform up. 2. **Platform subscription** - flat monthly, runs and maintains it. 3. **Per-active-client** - small fee per client they actively coach (countable in your system, no need to see their books).

03 3. The readiness finding (why this is closer than it looks)

The data layer is **already multi-tenant**. Every meaningful table (`clients` , readings, programs, nutrition, CRM/leads) carries `coach_id` with RLS (`coach_id = auth.uid()`). Data isolation between partners already exists. The hard part is done.

So the gap to sell is **not architecture**. It is: - **Layer 1 (engine)**: essentially a decision (commit to A). Near-zero build. - **Layer 2 (application)**: branding, onboarding, billing. This is where the work is.

The mountain is **de-hardcoding the branding** (~230 files say "Body Recode," emails are hardcoded `Kade at Body Recode <kade@bodyrecode.au>`), not building a SaaS.

04 4. Build plan (sequenced, with effort)

Effort: **S** = days, **M** = 1-2 weeks, **L** = weeks. ⚡ = quick win.

Phase 0 - Decide & verify (do first, mostly non-build)

- ⚡ Lock per-seat numbers (setup / sub / per-client). **S**
- One-page founding-partner agreement + IP licence (needs legal/Ange). **S**
- Confirm (A) doctrine + hard floors stay. **done (decision made)**
- Verify two unknowns: (a) is there any coach-onboarding flow today, or only your account? (b) can a coach author a plan from scratch, or only generate-then-edit? **S to investigate**

Phase 1 - Pilot-ready (onboard partner #1 hand-gloved)

Because data is already `coach_id` -scoped, you can onboard one coach now with a *targeted* branding override, before the full de-hardcode. - Manually provision a coach account + workspace. **S-M** - Targeted branding override in the most visible places only: from-email, logo, name on scorecard + portal + key emails. **M** - Run the first doctrine-aligned coach as a hand-gloved pilot. **(delivery, not build)**

Phase 2 - Product-ready (repeatable, self-serve-ish)

- `tenant_config` table: name, logo, colours, from-email, reply-to, domain, doctrine version, Stripe keys. **S-M**
- Tenant resolver in middleware (subdomain / custom domain to tenant) + config provider threaded app-wide. **M**

- **De-hardcode branding** across the ~230 files: route name/logo/from-address/colours through `tenant_config`. **L (the big one)**
- Email shell + from-address per tenant. **M**
- Coach signup/onboarding flow + coach settings UI (manage their own branding/config). **M**

Phase 3 - Billing

- Per-tenant Stripe (Stripe Connect, so their clients pay them). **M-L**
- Your billing of partners: setup + subscription + per-active-client metering. **M**

Phase 4 - Scale & (B)

- Optional per-tenant doctrine *parameters* (tone, thresholds) as a middle ground. **M**
- **B.** method-injection pipeline (IP-extraction to doctrine config). **L+, after the ten.**
- Manual from-scratch plan builder if Phase 0 finds it is generate-then-edit only. **S-M**

05 5. Smallest path to revenue

Phase 0 + Phase 1 only. Lock pricing, write the one-pager, provision one coach, do a targeted branding pass, onboard a doctrine-aligned founding partner hand-gloved. Everything in Phases 2-4 is what turns one hand-gloved partner into a repeatable product for the other nine.

06 6. Who the first partners are (under A)

Because partners run BR's doctrine (body-state, fat loss, Fat Map), the first full-platform partners should be **doctrine-aligned**: body-composition, fat-loss, performance, physique-adjacent coaches. The engine reads, they prescribe their craft on top. Practitioners whose method is far from body-state work (e.g. pure yoga/breath) may be better as **referral/funnel partners** first, unless they want to run a body-state offer alongside their core work.

07 7. Modality / discipline layer (new axis)

A third configurability axis, separate from branding and doctrine. The programming layer is currently strength / performance / body-composition (exercise library, sets/reps/RPE, Fat Map training limits). A yoga coach needs a different movement vocabulary: poses, flows, holds, breath ratios, sequences.

- **Reading (Layer 1) transfers.** A yoga client still has a body state, and the engine is strongest on recovery/regulation, which is yoga's home turf. The diagnosis works. Only the prescription tooling is the gap.
- **What changes (Layer 2):**
 - A yoga movement library (poses, flows, breath) alongside the strength exercise library.
 - A prescription schema for yoga (sequences, hold durations, breath ratios) vs sets/reps/RPE.
 - Sort doctrine constraints into universal vs strength-specific. Recovery/regulation floors stay; Fat Map training limits, RPE caps and barbell-library restrictions do not apply.
- **Architecture:** introduce a `modality` (discipline) concept. Strength = modality 1 (exists). Yoga = modality 2 (build). Each tenant is assigned a modality that selects their library, prescription schema and applicable constraints. **Effort: L.**
- **Sequencing:** a yoga coach is a heavier first partner than a same-modality (body-comp/performance) coach, who could use the program engine as-is. Either keep the first founding partners same-modality and build the yoga modality deliberately when committing to Melisa as a platform partner, or scope the yoga modality as its own mini-project up front if she is the priority pilot.

08 8. Open items to confirm

- Coach onboarding: exists or net-new?
- Manual plan authoring: from scratch, or generate-then-edit?
- From-email/branding constants: how centralised? (sizes the de-hardcode job)
- Program engine: how tightly coupled to the strength modality? (sizes the yoga-modality build)